

Oppgave 2

2.1

I denne delen ble det laget en graf basert på insidenslister. Både noder og kanter er representert som egne objekter, og hver node inneholder en liste over kantene den er insident til.

Hver node har en unik strengverdi som identifikator, mens kantene kan ha ikke unike etiketter. Funksjonen `insert_edge` sørger for å opprette både noder og kanter ved behov, og legger til kantene i de relevante listene.

2.2

Programmet støtter innlesing og lagring av grafdata fra og til fil. Filformatet består av linjer med tre elementer: startnode, kantetikett og målnode.

Ved innlesing opprettes grafen ved å kalle `insert_edge` for hver linje i filen. Tilsvarende kan grafen skrives tilbake til fil i samme format.

2.3

Det ble definert en abstrakt klasse `Graph` som spesifiserer grensesnittet for grafoperasjoner. Denne klassen inneholder rene virtuelle funksjoner som må lages av alle konkrete grafklasser.

`ListGraph` ble gjort til en subklasse av `Graph`, slik at alle operasjoner kan brukes gjennom det samme grensesnittet. Dette gjør det mulig å bruke samme algoritmer uavhengig av underliggende datastruktur.

2.4

Det ble laget en matrisebasert graf (`MatrixGraph`). Denne representasjonen bruker en todimensjonal matrise hvor hver celle inneholder en liste av kantetiketter mellom to noder.

I denne grafen er det kun noder som er objekter, mens kantene representeres indirekte gjennom matrisen. Dette gjør det mulig å håndtere flere kanter mellom samme par av noder.

2.5

Når en node slettes, fjernes også alle kanter som er koblet til den. Dette hindrer grafen i å havne i en inkonsistent tilstand. I en listebasert løsning sjekker vi også om det oppstår isolerte noder (noder uten kanter). Slike noder slettes hvis de ikke lenger er koblet til noe.

2.6

Kopikonstruktør og kopitilordning lager en egen kopi av grafens data. Flyttekonstruktør og flyttetilordning overfører eierskapet til ressursene. Destruktoren frigjør alle ressursene, og flytteoperasjonene er merket med «noexcept» som sier at funksjonen garanterer å ikke gi feil.

Oppgave 3

3.1

Tarjans algoritme er lagt til for å finne sterkt sammenhengende komponenter i grafen. Algoritmen benytter dybde-først-søk og holder styr på indekser og lavlink-verdier for hver node. Koden bruker Graph-grensesnittet, slik at samme algoritme kan kjøres på både ListGraph og MatrixGraph. Algoritmen har tidskompleksitet $O(V + E)$. ListGraph gir bedre ytelse fordi den kun itererer over eksisterende naboer, mens MatrixGraph må sjekke alle mulige noder.

3.2

Det er skrevet en algoritme for å identifisere par av noder som er knyttet sammen via to ulike stier med bestemte kantetiketter. Spørringen leses fra en fil som inneholder to sekvenser av etiketter. For hver node utføres et søk langs begge stiene, og resultatene sammenlignes for å finne felles målnoder. Algoritmen har tilnærmet kompleksitet $O(n * m * d)$, der d er gjennomsnittlig grad. Manglende caching gjør at samme naboer beregnes flere ganger. En mulig optimalisering ville vært å indeksere naboer per kantetikett for å redusere antall gjentatte oppslag.

3.3

Makefile er endret slik at programmet kan brukes ved hjelp av følgende kommandoer:

```
make pro22  
make pro25  
make pro31  
make pro32
```

Disse kommandoene kjører ulike deler av programmet.

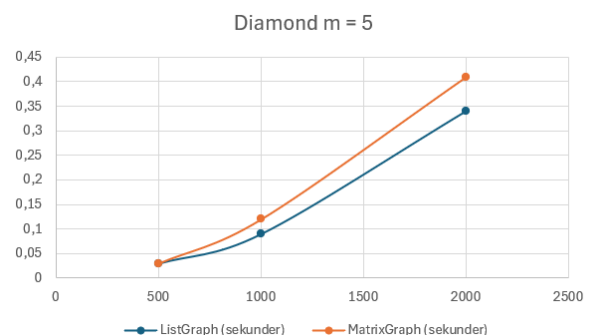
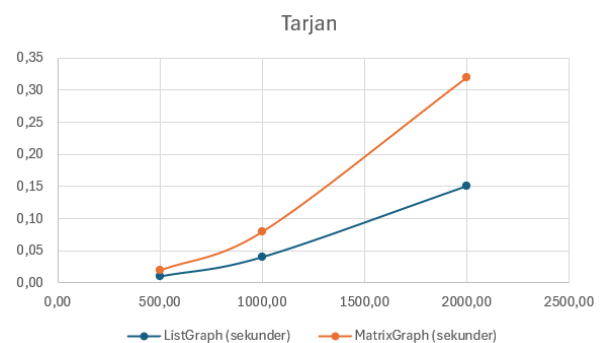
3.4 Kjøretidsmåling

Test 1, SCC (Tarjan):

n	ListGraph (sekunder)	MatrixGraph (sekunder)
500	0.01	0.02
1000	0.04	0.08
2000	0.15	0.32

Test 2, Diamond (m = 5):

n	ListGraph (sekunder)	MatrixGraph (sekunder)
500	0.03	0.03
1000	0.09	0.12
2000	0.34	0.41



Test 3, Diamond (Ulik m):

n	m	ListGraph (sekunder)	MatrixGraph (sekunder)
500	5	0.01	0.03
1000	10	0.04	0.13
2000	20	0.21	0.54

Resultatene viser at ListGraph skalerer bedre enn MatrixGraph, spesielt for større grafer. Dette skyldes at matrisebasert representasjon har høyere kostnad ved nabooppslag. For diamantproblemet øker kjøretiden raskere når stilengden m øker.

3.5 ER-Diagram

Den nedre delen viser grafens datastruktur som et ER-diagram. Den øvre delen viser klassestrukturen.

